Case Study 2 AI Based Legal Decision Support System

Case Based Reasoning System Development

Ezekiel M. Ibasco | AN42 | Individual Implementation

System Description

This academic prototype supports legal case preparation by comparing a new alleged cybercrime with previous cases. It performs the complete Case-Based Reasoning cycle: retrieve, reuse, revise, and retain. The selected scenario is theft involving an electronic transfer. The system is an educational decision-support example and is not legal advice.

Case Base

Five past cases are stored as nested dictionaries. Each problem describes the offense, channel, evidence, intent, loss, and prior-record status. Each solution contains argument patterns, recommended actions, and a recorded outcome. The examples include unauthorized transfers, online theft, digital fraud, physical theft, and phishing-related fraud.

Similarity Method

The weighted similarity score assigns 30 percent to offense type, 20 percent to transaction channel, 25 percent to evidence overlap, 10 percent to intent, 10 percent to financial loss, and 5 percent to prior-record status. Evidence overlap uses Jaccard similarity and financial loss uses a normalized numerical comparison.

Adaptation Logic

The closest case supplies the initial strategy. The system then adds recommendations for authenticating electronic evidence, tracing funds, connecting account and device activity to the responsible person, verifying the loss, and considering the absence of a prior record when legally relevant.

Individual Reflection

I completed the case representation, similarity function, adaptation rules, and retention stage individually. The main challenge was converting qualitative legal similarities into numerical values without hiding how the score was produced. I addressed this by assigning explicit weights and using Jaccard similarity for evidence overlap. I learned that CBR does not simply copy an old answer because the retrieved solution still has to be revised for the new facts. I also learned that retaining a solved case allows the knowledge base to grow over time. Testing different cases showed why feature selection and weighting strongly affect retrieval quality.

Complete Python Program

```
# CASE STUDY 2: AI-BASED LEGAL DECISION SUPPORT SYSTEM
# Academic Case-Based Reasoning prototype; not legal advice.

from copy import deepcopy

# ------------------------ CASE REPRESENTATION ------------------------
case_base = [
    {
        'case_id': 'LC-001',
        'title': 'Unauthorized Mobile Banking Transfer',
        'problem': {
            'offense_type': 'unauthorized bank transfer',
            'channel': 'electronic transfer',
            'evidence': {'transaction logs', 'device records', 'account records'},
            'intent': 'high',
            'loss': 220000,
            'prior_record': False,
        },
        'solution': {
            'arguments': [
                'Verify whether the accused authorized or controlled the transfer.',
                'Test whether the digital records reliably connect the accused to the transaction.',
            ],
            'recommended_actions': [
                'Obtain certified bank transaction and account-access logs.',
                'Preserve the relevant device and request a forensic examination.',
                'Trace the destination account and any subsequent fund movements.',
            ],
            'outcome': 'Digital evidence was admitted after authentication; liability depended on
identity attribution.',
        },
    },
    {
        'case_id': 'LC-002',
        'title': 'Online Account Takeover and Theft',
        'problem': {
            'offense_type': 'online theft',
            'channel': 'online account',
            'evidence': {'login logs', 'ip records', 'device records'},
            'intent': 'high',
            'loss': 180000,
            'prior_record': True,
        },
        'solution': {
            'arguments': [
                'Determine whether account access can be attributed to the accused rather than another
user.',
                'Examine whether unauthorized control and intent are supported by the access history.',
            ],
            'recommended_actions': [
                'Compare login times, IP records, and device identifiers.',
                'Review account recovery and password-reset events.',
            ],
            'outcome': 'Account and network records supported a finding of unauthorized access.',
        },
    },
    {
        'case_id': 'LC-003',
        'title': 'Digital Wallet Payment Fraud',
        'problem': {
            'offense_type': 'digital fraud',
            'channel': 'digital wallet',
            'evidence': {'transaction logs', 'messages', 'account records'},
            'intent': 'medium',
            'loss': 95000,
            'prior_record': False,
        },
        'solution': {
            'arguments': [
                'Assess whether the communications show deception and fraudulent intent.',
                'Establish who controlled the wallet receiving the disputed payment.',
            ],
            'recommended_actions': [
                'Preserve wallet transaction histories and related messages.',
                'Confirm account ownership through the service provider.',
            ],
            'outcome': 'The matter was resolved after wallet ownership and communications were verified.
```

```
',
        },
    },
    {
        'case_id': 'LC-004',
        'title': 'Physical Property Theft',
        'problem': {
            'offense_type': 'physical theft',
            'channel': 'in person',
            'evidence': {'witness statement', 'cctv footage'},
            'intent': 'medium',
            'loss': 30000,
            'prior_record': False,
        },
        'solution': {
            'arguments': [
                'Evaluate identification evidence and proof of unlawful taking.',
            ],
            'recommended_actions': [
                'Review original CCTV footage and interview witnesses separately.',
            ],
            'outcome': 'The decision relied primarily on corroborated physical evidence.',
        },
    },
    {
        'case_id': 'LC-005',
        'title': 'Phishing Based Transfer Fraud',
        'problem': {
            'offense_type': 'digital fraud',
            'channel': 'electronic transfer',
            'evidence': {'messages', 'transaction logs', 'ip records'},
            'intent': 'high',
            'loss': 260000,
            'prior_record': True,
        },
        'solution': {
            'arguments': [
                'Connect the deceptive communications and fund transfer to the same actor.',
                'Examine whether the communications demonstrate deliberate fraudulent intent.',
            ],
            'recommended_actions': [
                'Preserve message metadata and transfer records.',
                'Trace IP activity and recipient accounts.',
            ],
            'outcome': 'Correlated messages and financial records supported the prosecution theory.',
        },
    },
]

# The new problem specified in Case Study 2.
new_case = {
    'case_id': 'LC-006',
    'title': 'Theft Involving Electronic Transfer',
    'problem': {
        'offense_type': 'theft involving electronic transfer',
        'channel': 'electronic transfer',
        'evidence': {'transaction logs', 'device records', 'messages'},
        'intent': 'high',
        'loss': 250000,
        'prior_record': False,
    },
}

# ------------------------- RETRIEVE STAGE -------------------------
WEIGHTS = {
    'offense_type': 0.30,
    'channel': 0.20,
    'evidence': 0.25,
    'intent': 0.10,
    'loss': 0.10,
    'prior_record': 0.05,
}

CYBER_OFFENSES = {
    'unauthorized bank transfer',
    'online theft',
    'digital fraud',
    'theft involving electronic transfer',
}
DIGITAL_CHANNELS = {'electronic transfer', 'online account', 'digital wallet'}
```

```
INTENT_LEVEL = {'low': 0, 'medium': 1, 'high': 2}

def categorical_similarity(first, second, related_group):
    """Return 1 for exact values, 0.6 for related values, or 0 otherwise."""
    if first == second:
        return 1.0
    if first in related_group and second in related_group:
        return 0.6
    return 0.0

def jaccard_similarity(first_set, second_set):
    """Measure evidence overlap using intersection divided by union."""
    union = first_set | second_set
    return len(first_set & second_set) / len(union) if union else 1.0

def loss_similarity(first, second):
    """Give closer financial losses a higher score."""
    largest = max(first, second, 1)
    return 1.0 - abs(first - second) / largest

def intent_similarity(first, second):
    distance = abs(INTENT_LEVEL[first] - INTENT_LEVEL[second])
    return max(0.0, 1.0 - distance / 2)

def calculate_similarity(new_problem, past_problem):
    """Return the weighted similarity and its component scores."""
    components = {
        'offense_type': categorical_similarity(
            new_problem['offense_type'], past_problem['offense_type'], CYBER_OFFENSES
        ),
        'channel': categorical_similarity(
            new_problem['channel'], past_problem['channel'], DIGITAL_CHANNELS
        ),
        'evidence': jaccard_similarity(
            new_problem['evidence'], past_problem['evidence']
        ),
        'intent': intent_similarity(new_problem['intent'], past_problem['intent']),
        'loss': loss_similarity(new_problem['loss'], past_problem['loss']),
        'prior_record': float(
            new_problem['prior_record'] == past_problem['prior_record']
        ),
    }
    total = sum(components[name] * WEIGHTS[name] for name in WEIGHTS)
    return total, components

def retrieve_cases(case_base, new_problem):
    ranking = []
    for past_case in case_base:
        score, components = calculate_similarity(new_problem, past_case['problem'])
        ranking.append({
            'case': past_case,
            'score': score,
            'components': components,
        })
    return sorted(ranking, key=lambda item: item['score'], reverse=True)

# ------------------------- REUSE AND REVISE STAGES -------------------------
def add_unique(items, new_item):
    if new_item not in items:
        items.append(new_item)

def adapt_solution(retrieved_case, target_case):
    """Reuse the closest solution and revise it for the new technical context."""
    adapted = deepcopy(retrieved_case['solution'])
    problem = target_case['problem']

    add_unique(
        adapted['arguments'],
        'Require authentication, integrity, and a documented chain of custody for electronic evidence.',
    )
    add_unique(
        adapted['arguments'],
        'Distinguish ownership of the account or device from proof that the accused initiated the
transfer.',
    )
    add_unique(
        adapted['recommended_actions'],
        'Correlate transaction timestamps with device, account-access, and message records.',
    )
    if problem['channel'] == 'electronic transfer':
```

```
        add_unique(
            adapted['recommended_actions'],
            'Trace the electronic transfer through recipient and intermediary accounts.',
        )
    if problem['loss'] >= 200000:
        add_unique(
            adapted['recommended_actions'],
            'Verify the documented amount of loss and preserve records of any recovered funds.',
        )
    if not problem['prior_record']:
        add_unique(
            adapted['arguments'],
            'If legally relevant, present the absence of a prior record as a mitigating fact.',
        )
    adapted['outcome'] = 'Pending court result; update this field after review or final verdict.'
    adapted['adapted_from'] = retrieved_case['case_id']
    return adapted

# ------------------------- RETAIN STAGE -------------------------
def retain_case(case_base, solved_case):
    """Store a newly solved case unless its identifier already exists."""
    if any(case['case_id'] == solved_case['case_id'] for case in case_base):
        raise ValueError(f"Case {solved_case['case_id']} already exists.")
    case_base.append(deepcopy(solved_case))

# ------------------------- CLEAR PROGRAM OUTPUT -------------------------
def print_problem(case):
    problem = case['problem']
    print(f"Case ID: {case['case_id']}")
    print(f"Title: {case['title']}")
    print(f"Offense: {problem['offense_type']}")
    print(f"Channel: {problem['channel']}")
    print(f"Evidence: {', '.join(sorted(problem['evidence']))}")
    print(f"Intent level: {problem['intent']}")
    print(f"Alleged loss: PHP {problem['loss']:,.2f}")
    print(f"Prior record: {'Yes' if problem['prior_record'] else 'No'}")

def print_numbered(items):
    for number, item in enumerate(items, start=1):
        print(f'  {number}. {item}')

print('=' * 72)
print('AI-BASED LEGAL DECISION SUPPORT SYSTEM')
print('Academic CBR prototype - not legal advice')
print('=' * 72)
print('\nNEW PROBLEM DESCRIPTION')
print_problem(new_case)

ranking = retrieve_cases(case_base, new_case['problem'])
print('\nRETRIEVE STAGE - SIMILARITY RANKING')
for rank, result in enumerate(ranking, start=1):
    past_case = result['case']
    print(
        f"{rank}. {past_case['case_id']} - {past_case['title']}: "
        f"{result['score'] * 100:.2f}%"
    )

best_match = ranking[0]['case']
best_score = ranking[0]['score']
print('\nMOST RELEVANT PAST CASE RETRIEVED')
print(f"{best_match['case_id']} - {best_match['title']} ({best_score * 100:.2f}%)")
print('Similarity components:')
for attribute, component_score in ranking[0]['components'].items():
    print(f"  {attribute.replace('_', ' ').title()}: {component_score * 100:.2f}%")

adapted_solution = adapt_solution(best_match, new_case)
new_case['solution'] = adapted_solution

print('\nREUSE AND REVISE STAGES - ADAPTED FINAL SOLUTION')
print(f"Adapted from: {adapted_solution['adapted_from']}")
print('Argument patterns:')
print_numbered(adapted_solution['arguments'])
print('Recommended actions:')
print_numbered(adapted_solution['recommended_actions'])
print(f"Outcome status: {adapted_solution['outcome']}")

retain_case(case_base, new_case)
print('\nRETAIN STAGE')
print(f"New case saved: {new_case['case_id']} - {new_case['title']}")
print(f'Updated case count: {len(case_base)}')
```

```
print('Updated case base:')
for stored_case in case_base:
    print(f"  {stored_case['case_id']}: {stored_case['title']}")
```

Program Output

```
================================================================================
AI-BASED LEGAL DECISION SUPPORT SYSTEM
Academic CBR prototype - not legal advice
================================================================================

NEW PROBLEM DESCRIPTION
Case ID: LC-006
Title: Theft Involving Electronic Transfer
Offense: theft involving electronic transfer
Channel: electronic transfer
Evidence: device records, messages, transaction logs
Intent level: high
Alleged loss: PHP 250,000.00
Prior record: No

RETRIEVE STAGE - SIMILARITY RANKING
1. LC-001 - Unauthorized Mobile Banking Transfer: 74.30%
2. LC-005 - Phishing Based Transfer Fraud: 70.12%
3. LC-003 - Digital Wallet Payment Fraud: 56.30%
4. LC-002 - Online Account Takeover and Theft: 52.20%
5. LC-004 - Physical Property Theft: 11.20%

MOST RELEVANT PAST CASE RETRIEVED
LC-001 - Unauthorized Mobile Banking Transfer (74.30%)
Similarity components:
  Offense Type: 60.00%
  Channel: 100.00%
  Evidence: 50.00%
  Intent: 100.00%
  Loss: 88.00%
  Prior Record: 100.00%

REUSE AND REVISE STAGES - ADAPTED FINAL SOLUTION
Adapted from: LC-001
Argument patterns:
  1. Verify whether the accused authorized or controlled the transfer.
  2. Test whether the digital records reliably connect the accused to the transaction.
  3. Require authentication, integrity, and a documented chain of custody for electronic
evidence.
  4. Distinguish ownership of the account or device from proof that the accused initiated the
transfer.
  5. If legally relevant, present the absence of a prior record as a mitigating fact.
Recommended actions:
  1. Obtain certified bank transaction and account-access logs.
  2. Preserve the relevant device and request a forensic examination.
  3. Trace the destination account and any subsequent fund movements.
  4. Correlate transaction timestamps with device, account-access, and message records.
  5. Trace the electronic transfer through recipient and intermediary accounts.
  6. Verify the documented amount of loss and preserve records of any recovered funds.
Outcome status: Pending court result; update this field after review or final verdict.

RETAIN STAGE
New case saved: LC-006 - Theft Involving Electronic Transfer
Updated case count: 6
Updated case base:
  LC-001: Unauthorized Mobile Banking Transfer
  LC-002: Online Account Takeover and Theft
  LC-003: Digital Wallet Payment Fraud
  LC-004: Physical Property Theft
  LC-005: Phishing Based Transfer Fraud
  LC-006: Theft Involving Electronic Transfer
```